

 **RELIABILITY**

Model Fallback and Circuit Breaker Patterns

When your primary LLM provider goes down, your users should still get an answer. When a provider is throttling you, you should not hammer it with retries. These two patterns make your LLM pipeline production-grade.

What you will build: full resilience stack

 **CIRCUIT BREAKER**

 **FALLBACK CHAIN**

 **TENACITY RETRY**

 **BULKHEAD**

Production Python code. Redis state. LiteLLM Router. Real failure scenarios.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 **Repost**

Why LLM Pipelines **Need Resilience Patterns**

LLM APIs are external services with real-world failure modes. OpenAI, Anthropic, and Google Cloud all have incidents. Your production system must survive them with zero user-facing errors.

🚫 What Can Fail

Rate limit (429) errors during traffic spikes. Provider outages (OpenAI had 14 incidents in 2024). Timeout errors on long generation requests. Quota exhaustion on monthly limits. Model deprecations with no notice.

⚠️ Without Resilience Patterns

Every provider failure becomes a user-visible 500 error. Retry storms hit the already-struggling provider making the outage worse. Queue backlogs build up. Revenue impact scales with downtime.

🛡️ With Resilience Patterns

Failures are transparent to users. The circuit breaker stops retry storms. Fallbacks to Anthropic or Azure OpenAI kick in automatically. Graceful degradation returns a cached or simplified response as last resort.



Circuit Breaker 3 States

The circuit breaker has three states that control whether requests are allowed through to the provider. This prevents retry storms and gives failing providers time to recover.

✓ CLOSED

Normal operation. All requests pass through. Failure counter tracks errors.

✖ OPEN

Provider is failing. All requests immediately fail-fast. No calls sent. Recovery timeout running.

🕒 HALF-OPEN

Recovery timeout expired. One probe request sent. Success: CLOSED. Fail: OPEN again.

⚙️ State Transitions

CLOSED to OPEN: failure_count exceeds threshold (default: 5 failures in 60 seconds).

OPEN to HALF-OPEN: recovery_timeout expires (default: 30 seconds).

HALF-OPEN to CLOSED: probe request succeeds.

HALF-OPEN to OPEN: probe request fails (start recovery timer again).

💡 The key insight: OPEN state stops cascade failures. Without it, 500 retries per second amplify the outage. With it, you send 0 requests until the provider recovers.



AI Coach John

Follow



↻ Repost

CircuitBreaker Redis-Backed Implementation

```

import redis, time

r = redis.Redis()

class CircuitBreaker:
    def __init__(self, name, threshold=5, timeout=30):
        self.name = name; self.threshold = threshold
        self.timeout = timeout

    def _state_key(self): return f"cb:{self.name}:state"
    def _fail_key(self):  return f"cb:{self.name}:failures"

    def is_open(self) → bool:
        state = r.get(self._state_key())
        return state == b"OPEN"

    def record_failure(self):
        count = r.incr(self._fail_key())
        r.expire(self._fail_key(), 60) # 60s window
        if count ≥ self.threshold:
            r.set(self._state_key(), "OPEN", ex=self.timeout)

    def record_success(self):
        r.delete(self._state_key(), self._fail_key())

```

🔑 Why Redis for State?

Redis shares circuit breaker state across all API worker pods. Without Redis, each pod has its own breaker and retries from all pods still hammer the failing provider. Redis makes the breaker cluster-wide.



LLMFallbackChain Multi-Provider Fallback

```
class LLMFallbackChain:
    def __init__(self):
        self.chain = [
            ("gpt-4o", CircuitBreaker("openai")),
            ("claude-sonnet-4-6", CircuitBreaker("anthropic")),
            ("gemini-1.5-pro", CircuitBreaker("google")),
        ]

    async def complete(self, messages: list) → str:
        for model, cb in self.chain:
            if cb.is_open():
                log(f"Skipping {model}: circuit OPEN")
                continue
            try:
                resp = await call_llm(model, messages)
                cb.record_success()
                return resp
            except (RateLimitError, APIError) as e:
                cb.record_failure()
                log(f"{model} failed: {e}. Trying next.")

        return graceful_degradation_response()
```



The chain tries providers in order, skipping any with an open circuit. The first successful response is returned. Only if all providers fail does graceful degradation kick in.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

Repost

LiteLLM Router with Fallback Config

LiteLLM provides a production-ready router with built-in fallbacks, retries, and load balancing across multiple providers. Drop-in replacement for the OpenAI client.

```
from litellm import Router

router = Router(model_list=[
    {"model_name": "gpt-4o",
     "litellm_params": {"model": "gpt-4o", "api_key": OPENAI_KEY}},
    {"model_name": "gpt-4o", # same logical name, second endpoint
     "litellm_params": {"model": "azure/gpt-4o", "api_key": AZURE_KEY}},
    {"model_name": "claude-fallback",
     "litellm_params": {"model": "anthropic/claude-sonnet-4-6"}},
],
 fallbacks=[{"gpt-4o": ["claude-fallback"]}],
 num_retries=3,
 retry_after=2
)

resp = await router.acompletion(
    model="gpt-4o",
    messages=[{"role": "user", "content": query}]
)
```

★ LiteLLM Advantages

Handles rate limits, timeouts, and fallbacks automatically. Unified API across 100+ providers. Built-in spend tracking. Zero code changes when switching providers.



Tenacity Retry with Exponential Backoff and Jitter

Naive retry hammers the provider. Exponential backoff with jitter spreads retries over time, preventing thundering-herd problems when many clients retry simultaneously.

```
from tenacity import (retry, stop_after_attempt,
    wait_exponential_jitter, retry_if_exception_type)
from openai import RateLimitError, APIConnectionError

@retry(
    stop=stop_after_attempt(4),
    wait=wait_exponential_jitter(
        initial=1, max=60, jitter=3
    ), # 1s, 2s+jitter, 4s+jitter, 8s+jitter...
    retry=retry_if_exception_type(
        (RateLimitError, APIConnectionError)
    ),
    reraise=True
)
async def call_with_retry(messages: list, model: str) → str:
    resp = await client.chat.completions.create(
        model=model, messages=messages
    )
    return resp.choices[0].message.content

# Do NOT retry on AuthenticationError or ContentFilterError
# Those will never succeed on retry
```



Bulkhead: Semaphore per Feature

The bulkhead pattern isolates features so one misbehaving feature cannot exhaust all concurrency slots. If the document summarizer runs 100 concurrent requests, it does not block the QA bot from getting any responses.

```
import asyncio

class BulkheadedLLMPool:
    def __init__(self, limits: dict[str, int]):
        # Separate semaphore per feature
        self.semaphores = {
            feature: asyncio.Semaphore(limit)
            for feature, limit in limits.items()
        }
        self.default_sem = asyncio.Semaphore(5)

    async def call(self, feature: str, messages: list) → str:
        sem = self.semaphores.get(feature, self.default_sem)
        async with sem:
            return await call_with_retry(messages, "gpt-4o")

pool = BulkheadedLLMPool({
    "summarizer": 10, # max 10 concurrent
    "qa_bot": 20,
    "code_assistant": 5,
})
```



Set concurrency limits based on typical usage and the rate limit headroom available for each feature. A bulkhead prevents "noisy neighbor" failures in shared LLM pools.



Graceful Degradation Last-Resort Responses

When all providers are unavailable, never return a raw 500 error to users. Return a contextually appropriate fallback response that lets them know you are working on it, with zero stack traces visible to the end user.

</> Degradation Response Factory

```
FALLBACK_MESSAGES = {
    "qa_bot":      "Our AI assistant is temporarily"
                  " unavailable. Try again in 2 min.",
    "summarizer":  "Summarization is temporarily down."
                  " Your document is safe.",
    "default":     "Our AI service is experiencing"
                  " issues. Please try again shortly.",
}

def graceful_degradation_response(feature="default") → str:
    msg = FALLBACK_MESSAGES.get(feature, FALLBACK_MESSAGES["default"])
    increment_degradation_counter(feature)
    return msg
```

🕒 Cached Response Fallback

For the QA bot, store the last successful answer for each of the top 100 frequent queries. During outages, return the cached answer with a "Note: using cached response from [date]" prefix. Better than a generic error message.



Prometheus Metrics for Circuit Breaker

```
from prometheus_client import Counter, Gauge, Histogram

llm_requests = Counter("llm_requests_total",
    "Total LLM calls", ["provider", "status"])

circuit_state = Gauge("llm_circuit_state",
    "Circuit state: 0=closed,1=open,2=half", ["provider"])

fallback_count = Counter("llm_fallbacks_total",
    "Fallback triggers", ["from_provider", "to_provider"])

degraded_responses = Counter("llm_degraded_responses_total",
    "Graceful degradation responses", ["feature"])

retry_histogram = Histogram("llm_retry_attempts",
    "Number of retries before success",
    buckets=[0, 1, 2, 3, 4])

# Example alert rule in Prometheus:
# alert: LLMCircuitOpen
# expr: llm_circuit_state{state="open"} > 0
# for: 30s
```

Key Dashboard

Circuit state per provider as a status panel. Fallback rate as a time series. Degraded response rate as an alert.

Alert Thresholds

Alert: any circuit in OPEN state for more than 30 seconds. Page: degraded response rate exceeds 5% for 2 minutes.



AI Coach John

Follow



 Repost



Resilience Patterns Comparison

Pattern	Problem Solved	Implementation	Overhead
Circuit Breaker	Stops retry storms on outages	Redis state machine	1 Redis read per call
Fallback Chain	Provider-level redundancy	LLMFallbackChain loop	Second call latency
Retry + Backoff	Transient errors (429, 502)	Tenacity decorator	Variable (per retry)
Bulkhead	Feature isolation	asyncio.Semaphore	Near-zero
LiteLLM Router	All of the above unified	Router config	Moderate (SDK layer)



Recommended Implementation Order

1. Start with LiteLLM Router (handles retries and fallbacks out of the box).
2. Add Redis circuit breaker for cluster-wide state sharing.
3. Add Bulkhead if you have feature isolation requirements.
4. Add graceful degradation messages for each feature.



Circuit Breaker Mistakes

- ✖ **In-process circuit breaker state only.** Each API pod has its own breaker. Pod A opens its breaker but Pod B keeps hammering the provider. Use Redis for shared state across all pods.
- ✖ **Retrying on errors that will never succeed.** `AuthenticationError`, `ContentFilterError`, and `InvalidRequestError` will not succeed on retry. Only retry transient errors: `RateLimitError` and connection errors.
- ✖ **Fixed retry delay instead of exponential backoff.** If 100 clients all retry at $t+1s$, the provider gets hit by all 100 at once again. Jitter and exponential backoff spread the load.
- ✖ **No bulkhead between features.** One feature with a traffic spike blocks all other features from getting concurrency slots. Each feature needs its own semaphore limit.
- ✖ **Raw exception stack trace returned to users.** Users should see a friendly degradation message. Log the full trace internally, never expose it externally.
- ✖ **No metrics on fallback rate.** Without tracking how often fallbacks trigger, you cannot detect a provider degrading gradually (before a full outage) from the data.



Resilience Patterns **Full Reference**

Component	Code	Key Config
Circuit breaker state	Redis SET with ex=timeout	threshold=5, timeout=30s
Fallback chain	LLMFallbackChain loop	3 providers minimum
Retry policy	tenacity @retry decorator	4 attempts, jitter=3s
Bulkhead limits	asyncio.Semaphore per feature	10-20 per feature
Managed routing	LiteLLM Router fallbacks config	num_retries=3
Graceful degradation	FALLBACK_MESSAGES dict	Per-feature messages
Observability	Prometheus circuit_state Gauge	Alert on OPEN state 30s+

 **Fail fast. Recover fast. Degrade gracefully. Never expose failures to users.**



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

 KEY TAKEAWAY

Every Provider Will Fail Eventually. Be Ready.

Production reliability is not about preventing failures. It is about recovering from them faster than users notice. Circuit breakers, fallback chains, and graceful degradation together give you a system that self-heals on any provider outage.

Minimum Viable Resilience Stack

- LiteLLM Router with 2 provider fallbacks configured before launch day.
- Redis circuit breaker shared across all API pods. 5 failures in 60s triggers OPEN state.
- Prometheus alert firing when any circuit stays OPEN for 30 seconds or more.

Follow AI Coach John for more production AI engineering content.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost